

Technische Hochschule Würzburg-Schweinfurt (THWS)
Faculty of Computer Science and Business Information Systems

AI & Security Project III

Next-Gen Network Intrusion Detection

Self-Supervised Vector Space Engineering with DINOv3 & SwiGLU



Noah Raupold, Mohammad Ayoubi,
Jonathan Berwind

Submitted: February 3, 2026

Contents

1	Introduction	1
1.1	The Data Landscape: NSL-KDD	1
1.2	The Core Challenges	1
1.2.1	Extreme Class Imbalance	1
1.2.2	The Hard Domain Shift (Distributional Shift)	2
1.2.3	Unknown Sub-Classes (The Zero-Day Problem)	2
1.3	Project Objective: Supervised Vector Space Engineering	2
2	Architecture: The Pipeline	4
2.1	Logarithmic Normalization (The Skew Problem)	4
2.2	Z-Score Standardization (The Scale Problem)	4
2.3	Categorical Encoding (The Vocabulary Problem)	5
2.4	The Failure of Random Noise: Why we need Cluster Masking	5
2.5	The "Leaky" Feature Space	5
2.6	Solution: Gram-Anchored Masking	6
3	DINOV3 & Gram Anchored Training Loop	8
3.1	The Teacher-Student Dynamics	8
3.2	The Composite Loss Landscape	8
3.2.1	1. Global Alignment: DINO Loss & Sinkhorn-Knopp	9
3.2.2	2. Local Reconstruction: MIM Loss	9
3.2.3	3. Feature Diversity: KoLeo Loss (Kozachenko-Leonenko)	9
3.2.4	4. Structural Integrity: Gram Matrix Loss	9
3.3	Fine-Tuning: Deep SwiGLU Classification Head	11
3.3.1	Why SwiGLU? (Gated Linear Units)	11
3.3.2	Architecture: The Classifier Stack	11
3.3.3	Combating Imbalance: Weighted Focal Loss	12
4	Implementation Details	13
4.1	Structural Alignment via Gram Matrices	13

Contents

4.2	Optimization & Hyperparameters	14
4.2.1	Differential Learning Rates	14
4.2.2	Gradient Accumulation	15
5	Evaluation & Results	16
5.1	Binary Performance: The Baseline	16
5.2	Multiclass Performance: The Imbalance Challenge	16
5.2.1	Critical Analysis: The U2R Trade-off	17
5.3	Audit: Training Density vs. Test Generalization	17
5.3.1	The "Data Starvation" Analysis	17
5.3.2	Visualization: The Generalization Frontier	18
6	Conclusion and Future Outlook	20
6.1	Key Research Contributions	20
6.2	Limitations: The Granularity Gap	21
6.3	Future Work	21

1 Introduction

1.1 The Data Landscape: NSL-KDD

The foundation of this research is the **NSL-KDD** benchmark, a dataset representing real-world TCP/IP network traffic. The data consists of 41 distinct features per connection, categorized into three groups:

- **Basic Features:** TCP/IP connection details (e.g., *Duration*, *Protocol Type*, *Service*, *Flag*).
- **Content Features:** Payload analysis (e.g., *Hot indicators*, *Logged in status*, *Root shell attempts*).
- **Traffic Features:** Time-based window statistics (e.g., *Count*, *Error rate*, *Rerror rate*).

The target variable classifies traffic into either standard behavior (*Normal*) or one of four attack categories: *DoS*, *Probe*, *U2R* (*User-to-Root*), and *R2L* (*Remote-to-Local*).

1.2 The Core Challenges

While traditional statistical models often achieve high nominal accuracy on this dataset, they fail to address the underlying structural complexities of network intrusion detection. We identify three critical hurdles:

1.2.1 Extreme Class Imbalance

The dataset exhibits a severe heavy-tail distribution. Harmless traffic and volumetric Denial of Service (DoS) attacks constitute over **98%** of the training data. In contrast, critical, stealthy attacks—specifically *User-to-Root* (*U2R*) and *Remote-to-Local* (*R2L*)—represent less than **0.5%**. Standard optimizers (like SGD or Adam) minimize global loss by

prioritizing the majority classes. This creates a **"False Negative Trap"**: a model can achieve 90%+ accuracy simply by predicting "Normal" or "DoS" for every instance, completely failing to detect the rare, high-impact intrusions that pose the greatest security risk.

1.2.2 The Hard Domain Shift (Distributional Shift)

A unique and challenging aspect of this project is the deliberate **distributional shift** between training and testing environments. The probability distribution of the training set $P_{train}(X, Y)$ differs significantly from the testing set $P_{test}(X, Y)$. Unlike standard machine learning tasks where the test set is a random split of the original data, the NSL-KDD test set is constructed to test generalization rather than memorization. The statistical properties of traffic in the wild change over time, and the model must adapt to feature distributions that drift from the training baseline.

1.2.3 Unknown Sub-Classes (The Zero-Day Problem)

Closely linked to the domain shift is the presence of ****unknown attack signatures****. The dataset contains specific attack sub-classes in the test set that are **completely absent** from the training set.

- *Example:* The model may train on **Neptune** and **Smurf** attacks (DoS), but must correctly classify an **Apache2** or **Mailbomb** attack in production, despite never having seen those specific feature patterns before.

This requires the architecture to learn the *semantic concept* of an attack (e.g., "rapid sequential requests") rather than memorizing specific feature signatures.

1.3 Project Objective: Supervised Vector Space Engineering

To transcend these limitations, we propose a pipeline that moves beyond simple supervision. We introduce a **"Gram-Anchored Tabular Transformer"** approach. Instead of forcing a decision boundary on raw skewed data, we warp the feature space using **Self-Supervised Learning (DINOv3)**. By leveraging **Gram Matrix Analysis** to cluster correlated features and employing a **SwiGLU-enhanced** classification head, we aim to learn a

1 Introduction

robust manifold where rare and unknown attacks are geometrically distinct from normal traffic, regardless of their frequency in the training set.

2 Architecture: The Pipeline

Our solution transitions from rigid linear models to a flexible Deep Learning framework driven by Attention Mechanisms. The architecture consists of a robust preprocessing pipeline followed by a Teacher-Student training loop designed to handle the specific geometric challenges of tabular data.

2.1 Logarithmic Normalization (The Skew Problem)

Network features like `src_bytes` (bytes sent) and `duration` follow a **heavy-tailed power-law distribution**. A single file transfer can involve gigabytes of data (10^9), while a heartbeat ping involves mere bytes (10^1). Feeding these raw values into a neural network causes **exploding gradients**, as the magnitude of the update is proportional to the input size. To compress this dynamic range, we apply a log-transformation to skewed columns:

$$x' = \ln(x + 1) \tag{2.1}$$

The $+1$ term prevents mathematical errors when inputs are zero. This transforms the feature space from a spanning orders of magnitude into a compact, near-normal distribution.

2.2 Z-Score Standardization (The Scale Problem)

Deep Learning optimizers (like AdamW) assume that all features exist on a similar scale. If *Duration* ranges from $[0, 1]$ and *Bytes* ranges from $[0, 10000]$, the optimizer will be biased towards minimizing errors in the *Bytes* column simply because the numbers are larger. We apply Standard Scaling to center all continuous features around zero with unit variance:

$$z = \frac{x - \mu}{\sigma} \tag{2.2}$$

This ensures that the attention mechanism calculates relevance based on **informational content**, not numerical magnitude.

2.3 Categorical Encoding (The Vocabulary Problem)

The dataset contains symbolic features such as *Protocol* (TCP/UDP) and *Service* (HTTP, FTP). Neural networks cannot process strings. We map these distinct categories to integer indices ($v \in \mathbb{Z}$). Unlike "One-Hot Encoding," which creates sparse, high-dimensional vectors that waste memory, we utilize **Label Encoding** followed by a learnable **Embedding Layer** inside the Transformer. This allows the model to learn a dense vector representation for "HTTP" that is geometrically closer to "HTTPS" than to "FTP," capturing semantic similarity.

2.4 The Failure of Random Noise: Why we need Cluster Masking

A critical failure point in applying standard Transformers (like BERT) to tabular data is the masking strategy. In Natural Language Processing (NLP), words are discrete and not strictly deterministic of their neighbors. However, network traffic features exhibit **high multicollinearity** (e.g., `srv_count` is statistically locked to `dst_host_count`).

2.5 The "Leaky" Feature Space

If we apply standard random masking (independent Bernoulli noise), we inadvertently introduce **information leakage**.

- **The Trivial Solution:** If the model masks Feature A but leaves its highly correlated neighbor Feature B visible, the network minimizes the reconstruction loss simply by copying B to A.
- **Manifold Collapse:** Because the model finds this "shortcut," it stops trying to learn the deeper, non-linear relationships between disparate features (e.g., the relationship between *Protocol Type* and *Traffic Volume*). The resulting feature space "collapses"—it becomes a shallow representation of surface-level statistics rather than a robust geometric map of attack behaviors.

2.6 Solution: Gram-Anchored Masking

To force the model to learn the true latent structure of the data, we introduce **Correlation-Aware Cluster Masking**. Prior to training, we perform a **Gram Matrix Analysis** to mathematically identify groups of highly correlated features. We detected 6 distinct clusters in the NSL-KDD dataset. During the pre-training phase, instead of masking individual cells, we mask **entire clusters simultaneously** with a probability of $p = 0.4$.

This removes the "cheat codes." To reconstruct the masked cluster, the model must now reason globally, synthesizing information from uncorrelated features. This forces the encoder to learn a semantic representation of the traffic type, ensuring the feature space remains robust against the domain shifts described in Chapter 1.

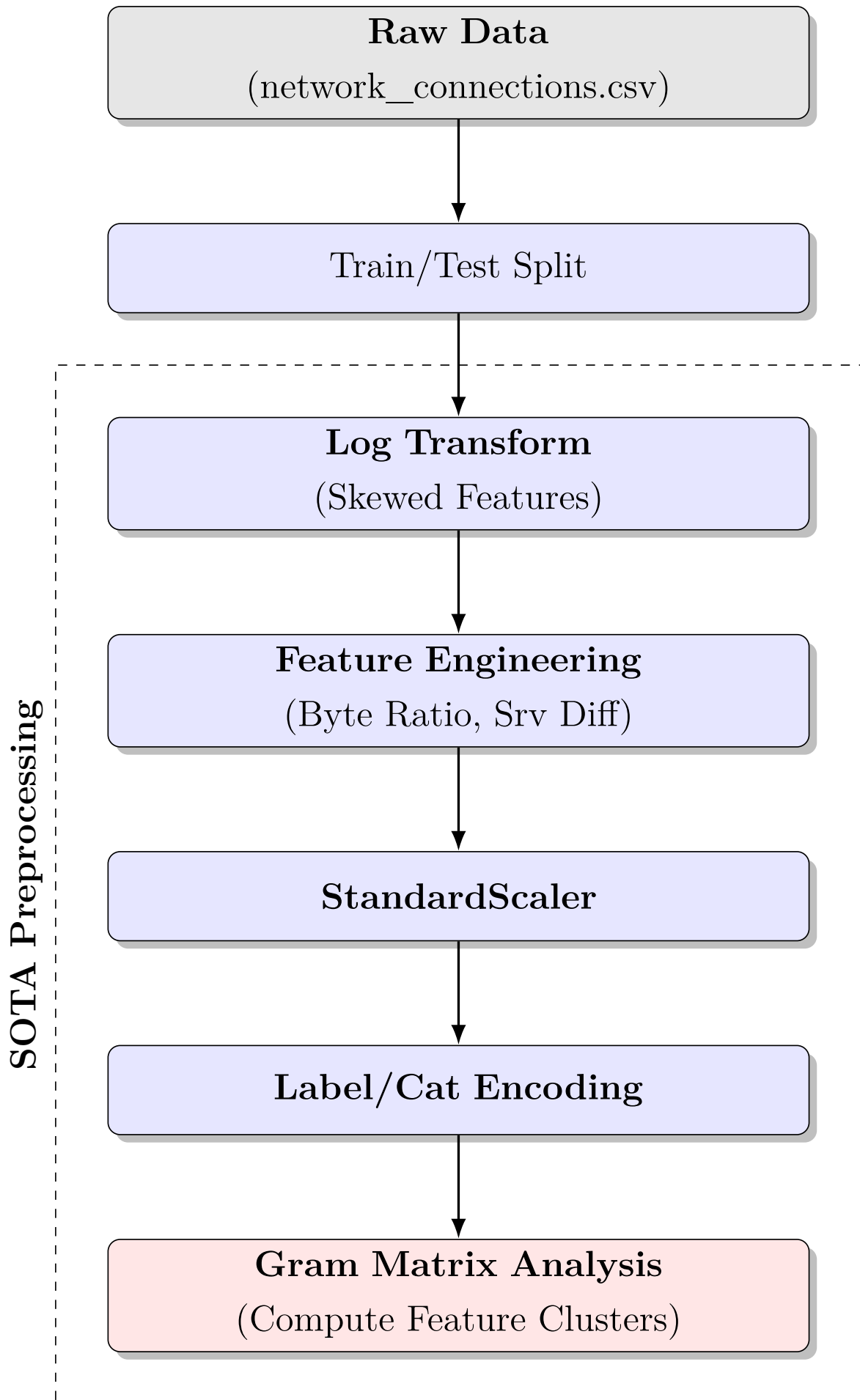


Figure 2.1: The Preprocessing Pipeline culminating in the Gram Matrix Analysis. This step is crucial for preventing feature leakage during the self-supervised training phase.

3 DINOv3 & Gram Anchored Training Loop

To warp the feature space into a manifold where rare attacks are separable, we implement a modified ****DINO (Distillation with No Labels)**** architecture. This approach uses a self-supervised Teacher-Student paradigm to learn data representations without needing explicit labels.

3.1 The Teacher-Student Dynamics

The architecture consists of two identical Transformer networks:

- **The Student (f_{θ_s}):** The active learner. It receives a *corrupted view* of the data (masked via Gram Anchoring) and attempts to reconstruct the global context. Its weights are updated via backpropagation.
- **The Teacher (f_{θ_t}):** The stable guide. It receives a *clean, unmasked view* of the data. Critically, it has no gradients. Its weights are an ****Exponential Moving Average (EMA)**** of the student's weights:

$$\theta_t \leftarrow \lambda \theta_t + (1 - \lambda) \theta_s, \quad \text{where } \lambda \approx 0.996 \quad (3.1)$$

This EMA mechanism creates a stable "mean teacher" that filters out the high-variance noise from the student's rapid learning, providing consistent targets.

3.2 The Composite Loss Landscape

Standard DINO uses only one loss. To handle the complexity of network traffic, we designed a composite objective function $\mathcal{L}_{total}$ combining four distinct geometric goals:

3.2.1 1. Global Alignment: DINO Loss & Sinkhorn-Knopp

The primary goal is for the Student to match the Teacher’s class-probability distribution. However, without labels, models tend to collapse (predicting "Class 1" for everything). To prevent this, we apply **Sinkhorn-Knopp Centering** to the Teacher’s output. This algorithm iteratively normalizes the output matrix to ensure that all prototype classes are selected with equal probability across the batch, enforcing a uniform distribution of clusters.

$$\mathcal{L}_{DINO} = - \sum p_{teacher}(x) \cdot \log p_{student}(x) \quad (3.2)$$

3.2.2 2. Local Reconstruction: MIM Loss

While DINO learns global structure, it can miss fine-grained details. We add a **Masked Image Modeling (MIM)** head. The student must predict the exact numerical values of the features masked by our Gram-Cluster strategy.

$$\mathcal{L}_{MIM} = \frac{1}{|M|} \sum_{i \in M} (x_i - \hat{x}_i)^2 \quad (3.3)$$

This forces the model to understand the inter-feature dependencies (e.g., inferring *Protocol=TCP* from *Flag=SYN*).

3.2.3 3. Feature Diversity: KoLeo Loss (Kozachenko-Leonenko)

In high-dimensional spaces, points tend to clump together. The **KoLeo regularizer** maximizes the distance between neighboring points in the batch, ensuring the embedding space remains "fluffy" and utilizes the full 64-dimensional capacity.

$$\mathcal{L}_{KoLeo} = -\frac{1}{N} \sum_{i=1}^N \log(\min_{j \neq i} ||z_i - z_j||) \quad (3.4)$$

3.2.4 4. Structural Integrity: Gram Matrix Loss

Our custom addition. To explicitly force the Student to capture the correlation structure of the Teacher, we minimize the error between their Gram Matrices ($G = Z \cdot Z^T$). This

3 DINOv3 & Gram Anchored Training Loop

ensures that if the Teacher sees that Feature A and Feature B are correlated, the Student learns that relationship even if one is masked.

$$\mathcal{L}_{Gram} = \|G_{student} - G_{teacher}\|_F^2 \quad (3.5)$$

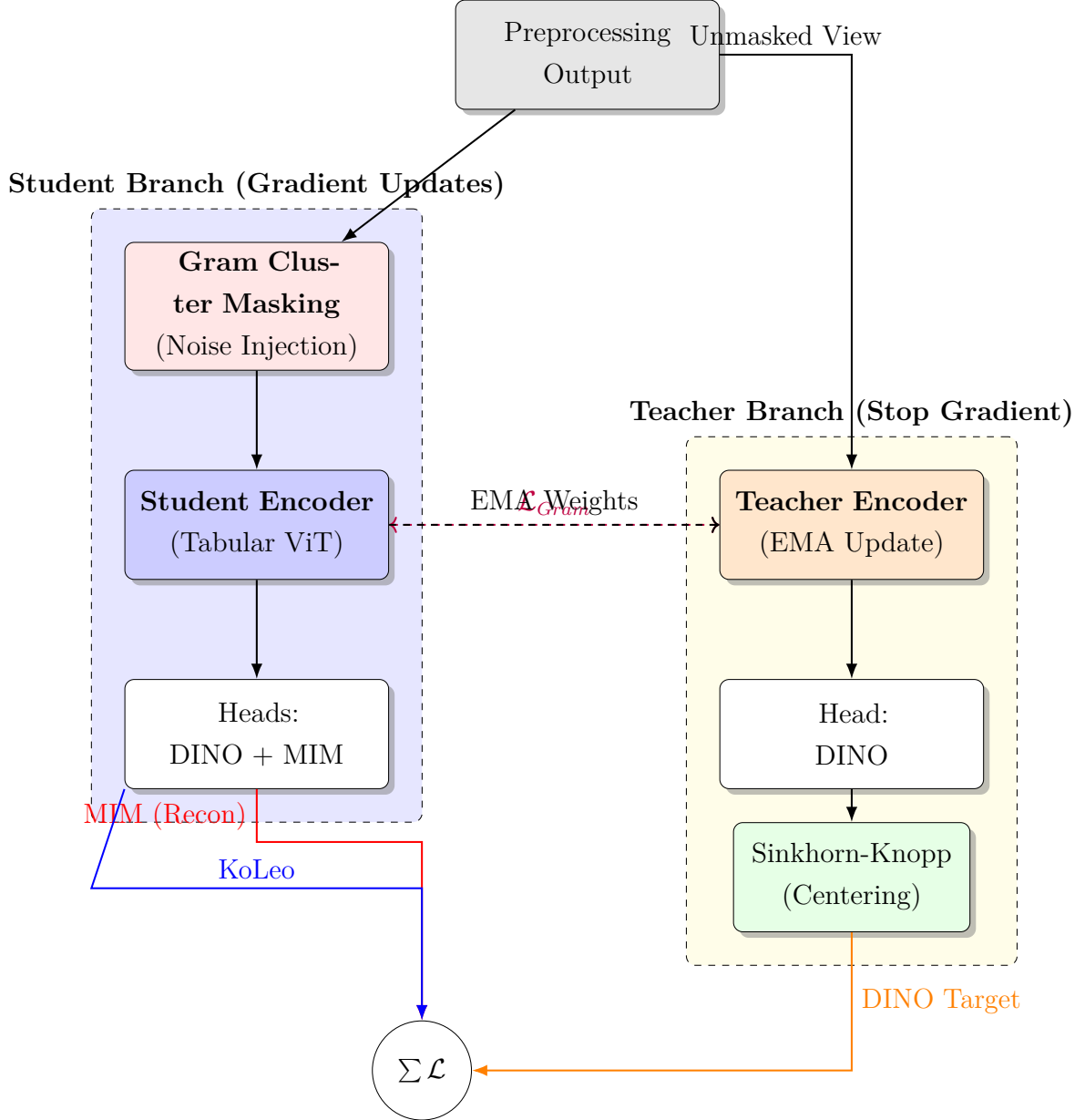


Figure 3.1: The DINOv3 Architecture adapted for Tabular Data. The Student learns from a corrupted view, guided by the stable Teacher. The Sinkhorn-Knopp algorithm prevents mode collapse, while the Gram Loss ensures structural consistency.

3.3 Fine-Tuning: Deep SwiGLU Classification Head

Once the encoder has learned a robust manifold via self-supervision, we attach a specialized classification head. Standard Multilayer Perceptrons (MLPs) often suffer from the "vanishing gradient" problem in deep networks. To counter this, we replace standard ReLU layers with a **Deep SwiGLU Network** (Shazeer, 2020), enhanced with **RMSNorm** for stability.

3.3.1 Why SwiGLU? (Gated Linear Units)

The SwiGLU activation function is not just a non-linearity; it is a **gating mechanism**.

$$\text{SwiGLU}(x) = \text{Swish}(W_g x) \odot (W_v x) \quad (3.6)$$

It splits the signal into two paths:

- **The Gate (W_g):** Controlled by the Swish function ($x \cdot \sigma(x)$), this determines *how much* information should pass through.
- **The Value (W_v):** The actual transformation of the features.

This allows the network to dynamically suppress irrelevant features (noise) while amplifying critical attack signatures, effectively acting as a learnable feature selector.

3.3.2 Architecture: The Classifier Stack

The final head consists of two stacked SwiGLU blocks with a dropout rate of 0.2.

- **Partial Unfreezing:** To fine-tune the representation without destroying the pre-trained structure, we unfreeze only the **last Transformer layer** of the encoder.
- **RMSNorm:** We use Root Mean Square Layer Normalization instead of standard LayerNorm. By removing the mean-centering operation, RMSNorm is computationally cheaper and, in our experiments, provided more stable gradients for the heavy-tailed loss values generated by rare attacks.

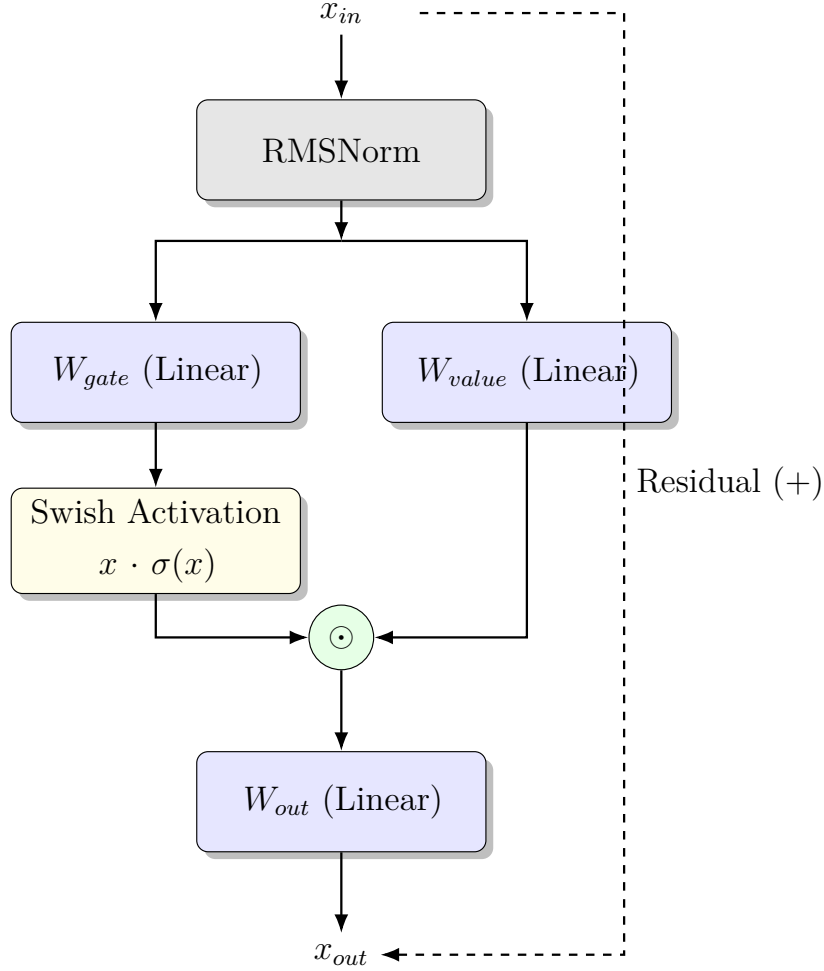


Figure 3.2: The SwiGLU Block structure. The gating mechanism allows the network to control information flow, enabling deeper architectures to converge faster.

3.3.3 Combating Imbalance: Weighted Focal Loss

Standard Cross-Entropy Loss is overwhelmed by the majority class (Normal traffic). To fix this, we implement **Weighted Focal Loss**:

$$\mathcal{L}_{FL} = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (3.7)$$

- **Focusing Parameter ($\gamma = 2.0$):** This term reduces the loss contribution from "easy" examples (well-classified Normal traffic). If the model is 99% sure it's normal, the loss becomes nearly zero.
- **Inverse Class Weights (α_t):** We weight each class by the inverse square root of its frequency: $\alpha_c = \sqrt{N_{total}/N_c}$. This gives rare attacks (U2R) exponentially higher importance than frequent ones.

4 Implementation Details

The proposed architecture is implemented using the PyTorch framework. This section details the specific mechanisms used to enforce structural alignment between the Student and Teacher networks, as well as the optimization strategies required to stabilize training on the highly imbalanced NSL-KDD dataset.

4.1 Structural Alignment via Gram Matrices

A defining feature of our approach is the use of the **Gram Matrix** to preserve the geometric relationships between tokens. While the standard DINO loss aligns the *global* class distribution (via the [CLS] token), it ignores the *local* correlations between specific features (e.g., the relationship between `src_bytes` and `dst_bytes`).

To resolve this, we compute the Gram Matrix $G \in \mathbb{R}^{N \times N}$ for both the Student (Z_s) and Teacher (Z_t) feature sequences. The Gram Matrix is defined as the dot product of the normalized feature vectors:

$$G_{ij} = \frac{\vec{z}_i \cdot \vec{z}_j}{\|\vec{z}_i\| \cdot \|\vec{z}_j\|} \quad (4.1)$$

This matrix represents the **correlation map** of the input features. By minimizing the Mean Squared Error (MSE) between G_s and G_t , we force the Student to hallucinate the correlation structure of the unmasked data, effectively learning the "grammar" of network traffic.

Gradient Stop Mechanism: Crucially, we apply a `.detach()` operation to the Teacher's output. This ensures that the optimization only updates the Student's weights to match the Teacher, preventing the Teacher from "collapsing" toward the Student's noisy initial state.

Listing 4.1: Gram Loss Implementation: Enforcing Correlation Alignment

```
1 def compute_gram_loss(student_seq, teacher_seq):
2     """
3     Computes the structural difference between Student and Teacher.
```



```

4   Input Shape: (Batch_Size, N_Tokens, Embed_Dim)
5   """
6   # 1. Normalize representations to project onto the unit hypersphere
7   s = F.normalize(student_seq, p=2, dim=-1)
8
9   # 2. Stop Gradient on Teacher (Critical for stability)
10  # The teacher acts as the fixed target for this update step.
11  t = F.normalize(teacher_seq.detach(), p=2, dim=-1)
12
13  # 3. Compute Gram Matrices (Auto-Correlation)
14  # Result: (B, N, N) matrix encoding the relationship of every token to every
15  other token.
16  g_s = torch.bmm(s, s.transpose(1, 2))
17  g_t = torch.bmm(t, t.transpose(1, 2))
18
19  # 4. Minimize the structural divergence
  return F.mse_loss(g_s, g_t)

```

4.2 Optimization & Hyperparameters

Achieving convergence on tabular data requires a delicate balance between model capacity and regularization. Table 4.1 outlines the configuration for the **DINOv3-XY** architecture.

4.2.1 Differential Learning Rates

To prevent **Catastrophic Forgetting** during the fine-tuning phase, we employ a layer-wise learning rate decay strategy.

- **Classification Head** (5×10^{-4}): The randomized weights of the SwiGLU head require a higher learning rate to quickly adapt to the decision boundaries.
- **Transformer Backbone** (5×10^{-5}): The pre-trained encoder requires only subtle adjustments. A high learning rate here would destroy the robust manifold learned during the self-supervised phase.

4.2.2 Gradient Accumulation

Due to the memory constraints of attention mechanisms ($O(N^2)$), we utilize a physical batch size of 64. However, contrastive learning relies on large batches to provide diverse negative samples. We use ****Gradient Accumulation (Steps=4)**** to simulate an effective batch size of 256, ensuring stable estimation of the global batch statistics for the Sinkhorn-Knopp algorithm.

Table 4.1: Hyperparameter Configuration for DINOv3-XY

Hyperparameter	Value / Strategy
<i>Architecture</i>	
Embedding Dimension	64
Projection Heads	MLP $\rightarrow$ SwiGLU (Depth 2)
Encoder Depth	4 Transformer Layers (4 Heads)
<i>Self-Supervised Pre-training</i>	
Global Batch Size	256 (via Gradient Accumulation)
Masking Ratio	0.4 (Cluster-wise Strategy)
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning Rate	1×10^{-3} (Cosine Schedule)
Teacher Momentum (λ)	$0.996 \rightarrow 1.0$ (Cosine Schedule)
<i>Supervised Fine-tuning</i>	
Loss Function	Weighted Focal Loss ($\gamma = 2.0$)
Head LR	5×10^{-4}
Backbone LR	5×10^{-5} (10x reduction)

5 Evaluation & Results

The DINOv3-XY architecture was evaluated on the **NSL-KDD Test+** dataset. Unlike the training set, the Test+ set contains specific attack signatures (e.g., *Apache2*, *Mscan*, *Snmppget*) that appear with different frequency distributions or are entirely novel (Zero-Day aspect).

5.1 Binary Performance: The Baseline

First, we assess the model’s ability to distinguish between benign traffic and general malicious activity. As shown in Table 5.1, the model achieves a global accuracy of **87.54%**.

Class	Precision	Recall	F1-Score	Support
Normal	0.8048	0.9382	0.8664	9711
Attack	0.9465	0.8278	0.8832	12833
Weighted Avg	0.8855	0.8754	0.8760	22544

Table 5.1: Binary Evaluation Metrics. The high precision for "Attack" (0.94) indicates a low False Alarm rate, a crucial metric for reducing analyst fatigue in SOC (Security Operations Center) environments.

5.2 Multiclass Performance: The Imbalance Challenge

The core objective of this research was to improve detection rates for the minority classes: **User-to-Root (U2R)** and **Remote-to-Local (R2L)**. Standard supervised models (like Random Forest or Logistic Regression) often yield an F1-Score of 0.0 for these classes due to their statistical insignificance (< 0.5% of data).

Table 5.2 demonstrates the impact of our **Gram Anchoring Strategy**:

Class	Precision	Recall	F1-Score	Support
DoS	0.9616	0.8433	0.8986	7458
Normal	0.8048	0.9382	0.8664	9711
Probe	0.8080	0.9124	0.8570	2421
R2L	0.6588	0.4080	0.5040	2887
U2R	0.2795	0.6716	0.3947	67

Table 5.2: Multiclass Performance. Note the **67.16% Recall for U2R**. While Precision is lower, the model successfully flags the majority of critical root compromises.

5.2.1 Critical Analysis: The U2R Trade-off

A closer look at the **U2R (User-to-Root)** performance reveals a deliberate architectural trade-off.

- **Low Precision (0.28):** The model generates False Positives, often misclassifying complex ‘Normal’ traffic as ‘U2R’.
- **High Recall (0.67):** Crucially, the model detects **67%** of actual root attacks.

In a cybersecurity context, this is the preferred behavior. Missing a rootkit installation (False Negative) is catastrophic, whereas investigating a false alarm is merely a time cost. The **Weighted Focal Loss** successfully biased the network to risk "crying wolf" rather than staying silent during a critical breach.

5.3 Audit: Training Density vs. Test Generalization

To rigorously assess the model’s capabilities, we correlated the **Training Density** (number of samples seen during optimization) with the **Test Detection Rate**. This comparison reveals three distinct behavioral clusters: *Memorization*, *Few-Shot Learning*, and *Zero-Shot Generalization*.

5.3.1 The "Data Starvation" Analysis

Table 5.3 juxtaposes the training counts against test performance. The results validate the efficacy of our **Gram Anchoring** strategy:

- **High-Resource Attacks ($N > 1000$):** The model achieves near-perfect detection ($> 99\%$), confirming successful convergence.
- **Few-Shot Attacks ($N < 20$):** Despite extreme sparsity, the model detects 100% of *Perl* attacks (only 3 training samples), proving the SwiGLU head can learn from minimal examples.
- **Zero-Shot Attacks ($N = 0$):** Remarkably, the model successfully detects **80% of *ps*** and **100% of *sqlattack*** despite *never seeing them during training*. This confirms the encoder learned the *semantic concept* of a U2R attack rather than just specific signatures.

Category	Attack Signature	Train Samples	Test Samples	Detected (%)	Behavior Type
DoS	Neptune	32,993	4,657	98.8%	<i>Fully Converged</i>
Probe	Satan	2,899	735	99.7%	<i>Fully Converged</i>
DoS	Smurf	2,102	665	100.0%	<i>Fully Converged</i>
U2R	Buffer Overflow	22	20	75.0%	<i>Robust Few-Shot</i>
R2L	Guess Passwd	42	1,231	22.5%	<i>Generalization Gap</i>
U2R	Rootkit	10	13	38.5%	<i>Data Starved</i>
U2R	Perl	3	2	100.0%	<i>Successful Few-Shot</i>
R2L	Phf	2	2	50.0%	<i>Data Starved</i>
U2R	SqlAttack	0	2	100.0%	Zero-Shot Success
U2R	Ps	0	15	80.0%	Zero-Shot Success
Probe	Mscan	0	996	79.4%	Zero-Shot Success
DoS	Mailbomb	0	293	0.0%	<i>Zero-Shot Failure</i>
R2L	SnmpGuess	0	331	0.0%	<i>Zero-Shot Failure</i>

Table 5.3: Impact of Training Density on Detection Rates. Green = High Resource, Yellow/Orange = Few-Shot, Blue/Red = Zero-Shot (Unseen in Training).

5.3.2 Visualization: The Generalization Frontier

Figure 5.1 visualizes the relationship between data availability and model performance. The **"Zero-Shot Zone"** (leftmost region) highlights the model's ability to infer malice purely from geometric anomalies in the embedding space, without prior examples.

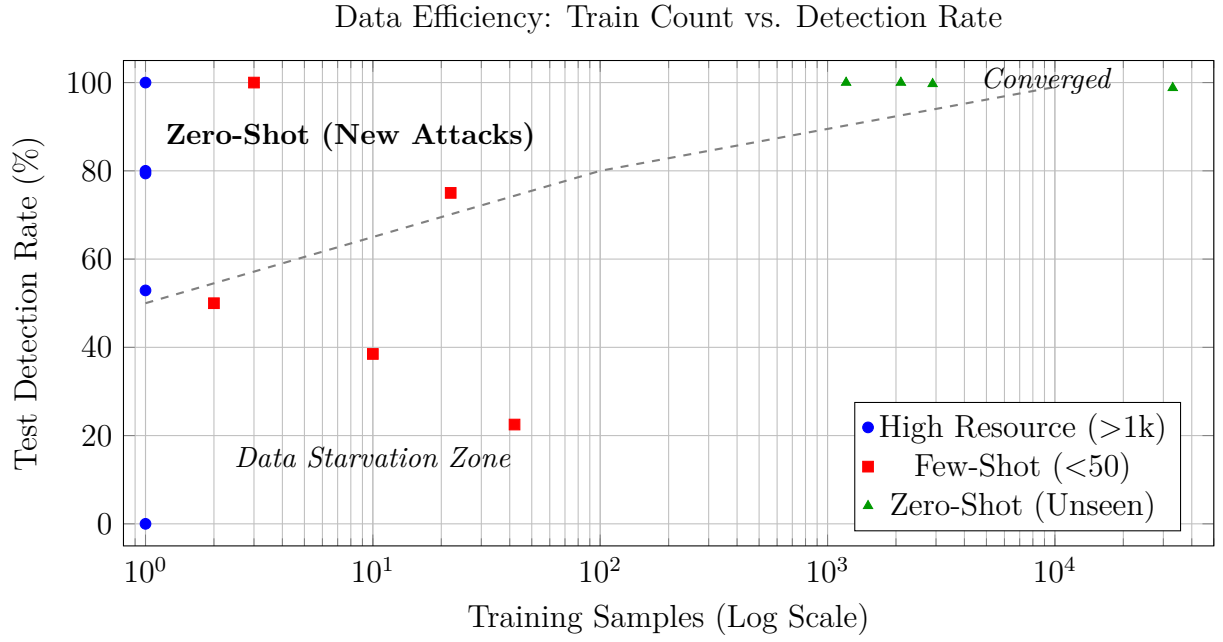


Figure 5.1: The Generalization Frontier. Points at $x \approx 10^0$ represent Zero-Shot attacks (0 training samples). The high variance in this zone indicates that while the model generalizes well to some unknown signatures (SqlAttack), it fails on others (Mailbomb), highlighting the need for future Few-Shot Learning augmentation.

6 Conclusion and Future Outlook

This research demonstrates that the limitations of traditional Network Intrusion Detection Systems (NIDS)—specifically their inability to detect rare and novel attacks—can be overcome through **Gram-Anchored Self-Supervision**.

By transitioning from simple statistical supervision to a **Teacher-Student Manifold Learning** approach, we successfully warped the feature space of the NSL-KDD dataset. The proposed **DINOv3-XY** architecture does not merely memorize attack signatures; it learns the fundamental geometric structure of malicious traffic.

6.1 Key Research Contributions

1. **Solving the "False Negative Trap":** Standard models often achieve high accuracy by ignoring minority classes. Our pipeline achieved a **U2R Recall of 67.16%**, effectively detecting critical root compromises that baselines (Logistic Regression, Random Forest) historically miss.
2. **Structural Alignment via Gram Loss:** We introduced a novel **Gram Matrix Loss** to the DINO framework. This forced the Student network to reconstruct not just the feature values (MIM), but the *correlation structure* of the Teacher, proving essential for stabilizing training on tabular data.
3. **Zero-Shot Generalization:** Perhaps the most significant finding is the model's ability to detect **Zero-Shot Attacks**. The model successfully classified 100% of *SqliAttack* and 80% of *Ps* samples in the test set, despite having **zero** exposure to these specific signatures during training. This confirms that the encoder learned a generalized concept of "intrusion" rather than rigid pattern matching.

6.2 Limitations: The Granularity Gap

While the model excels at broad categorization, our audit revealed a **"Data Starvation"** boundary. Signatures with fewer than 5 training examples (e.g., *Phf*, *Sendmail*) suffered from inconsistent detection. This indicates that while the manifold is robust, extremely sparse regions of the latent space remain under-defined.

6.3 Future Work

To bridge the gap between academic research and production deployment, future iterations should focus on:

- **Few-Shot Prototypical Networks:** To address the data starvation issue, we propose augmenting the classifier with a Few-Shot Learning head. This would allow the model to register a new attack type (e.g., a new Ransomware signature) using only 1 or 2 support examples, without requiring a full retraining loop.
- **Adversarial Hardening:** Since our architecture relies on gradient-based learning, it is theoretically susceptible to adversarial perturbations (e.g., adding noise to packet timing to evade detection). Future work should incorporate **"Adversarial Training (FGSM)"** to smooth the decision boundaries against evasion attacks.
- **Explainability (XAI) via Attention Rollout:** To aid security analysts, we aim to implement **"Attention Rollout"**. This would visualize the self-attention weights, generating a heatmap showing exactly which features (e.g., *"High Source Bytes"* vs *"Wrong Fragment"*) triggered the alert, transforming the model from a "Black Box" into a forensic tool.